

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ» (НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ, НГУ)

Факультет Механико-математический
Кафедра Программирования

Направление подготовки Математика и компьютерные науки

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Новиков Сергей Александрович

(Фамилия, Имя, Отчество автора)

Тема работы: Применение нейроэволюционных алгоритмов для
 моделирования когнитивных агентов

«К защите допущена»

Вр.и.о. зав. кафедрой,
к.ф.-м.н.

Емельянов П.Г. / _____
(фамилия, И., О.) (подпись, МП)

«....».....2026 г.

Научный руководитель

д.ф.-м.н.,
старший преподаватель
кафедры программирования

Пальянов А.Ю. / _____
(фамилия, И., О.) (подпись, МП)

«....».....2026 г.

Дата защиты: «....»2026 г.

Новосибирск, 2026

Содержание

1. Введение	6
2. Постановка задачи	6
2.1. Контекст исследования	6
2.2. Цель исследования	7
2.3. Задачи исследования	7
2.4. Метрики исследования	7
2.5. Математическая формализация	9
2.6. Сравнимые подходы	9
2.7. Ожидаемые результаты	10
3. Предшествующие работы	10
3.1. Глубокое обучение с подкреплением на основе аппроксимации Q-функции	10
3.1.1. Постановка задачи и мотивация	10
3.1.2. Основные идеи и архитектура метода	11
3.1.3. Результаты и выявленные ограничения	12
3.1.4. Связь с задачей моделирования когнитивных агентов	12
3.2. Эволюция нейронных сетей через расширение топологий .	13
3.2.1. Постановка задачи и мотивация	13
3.2.2. Основные идеи и архитектура NEAT	14
3.2.3. Связь с задачей моделирования когнитивного агента	15
4. Ход исследования	15

4.1.	Алгоритмический базис: Жизненный цикл NEAT	16
4.1.1.	Кодирование нейросети: генотип и фенотип	16
4.1.2.	Отказ от слоистой архитектуры в пользу произволь- ных графов	17
4.1.3.	Скрещивание и решение проблемы «соперничающих соглашений»	18
4.1.4.	Классификация генов при выравнивании	19
4.1.5.	Вычисление функции приспособленности	20
4.1.6.	Операторы эволюции: мутация и скрещивание	21
4.1.7.	Видообразование на практике	23
4.2.	Программная реализация и особенности NEAT-модели . . .	24
4.2.1.	Секция [NEAT]	24
4.2.2.	Секция [DefaultGenome]	25
4.2.3.	Секции [DefaultSpeciesSet] и [DefaultStagnation] . . .	25
4.2.4.	Секции [DefaultReproduction] и [DefaultStagnation] . .	25
4.3.	Условия проведения экспериментов	26
4.4.	Аппаратное обеспечение экспериментов	27
4.5.	Среда CartPole	28
4.5.1.	Пространство состояний	28
4.5.2.	Пространство действий	30
4.5.3.	Функция награды и условия завершения	30
4.5.4.	Конфигурация алгоритма NEAT для среды CartPole . .	31
4.5.5.	Архитектура и гиперпараметры агента DQN для сре- ды CartPole	33
4.6.	Среда LunarLander	35

4.6.1.	Пространство состояний	36
4.6.2.	Пространство действий	37
4.6.3.	Функция награды и условия завершения	37
4.6.4.	Конфигурация алгоритма NEAT для LunarLander . .	38
4.6.5.	Архитектура и гиперпараметры агента DQN для сре- ды LunarLander	40
4.7.	Среда Ant	41
4.7.1.	Пространство состояний и действий	42
4.7.2.	Функция награды и условия завершения	42
4.7.3.	Конфигурация алгоритма NEAT для среды Ant-v4 .	43
5.	Результаты исследования	46
5.1.	Результаты в среде CartPole	46
5.1.1.	Эффективность использования выборки и скорость схождения	47
5.1.2.	Качество решения и архитектурная сложность	48
5.1.3.	Устойчивость к внешним возмущениям	49
5.1.4.	Практические выводы	50
5.2.	Результаты в среде LunarLander	50
5.2.1.	Сравнительный анализ архитектурной сложности и обучения	50
5.2.2.	Феномен «нейролоботомии» и проблема локальных оптимумов	51
5.2.3.	Устойчивость в условиях неопределенности и внеш- них шумов	52

5.2.4.	Специфика программной реализации	53
5.2.5.	Практические выводы	53
5.3.	Результаты в среде Ant	54
5.3.1.	Обоснование выбора модальности обучения	54
5.3.2.	Эволюция функции вознаграждения и адаптация среды	55
5.3.3.	Практические выводы	56
6.	Направления дальнейших исследований	56
6.1.	Разработка гибридных мультимодальных моделей	56
6.2.	Интеграция алгоритмов в симулятор популяций	57

1. Введение

Обучение с подкреплением на данный момент является основным методом обучения агентов в условиях непрерывного принятия решений. Классические методы глубокого RL, такие как алгоритм DQN (Deep Q-Network), отлично это показывают. Однако и у него есть одна существенная проблема: инженеру необходимо заранее знать, какую архитектуру нейронной сети использовать для определенной задачи.

Обойти это ограничение можно с помощью метода нейроэволюции, в частности алгоритма NEAT (NeuroEvolution of Augmenting Topologies). Его основное отличие заключается в отказе от фиксированной архитектуры. Процесс обучения начинается с наименьшей топологии сети, которая усложняется только по мере необходимости. Это дает возможность оптимизировать структуру нейросети под нужды решения конкретной задачи.

В данной работе проводится сравнительный анализ классического глубокого Q-обучения и нейроэволюционного подхода.

2. Постановка задачи

2.1. Контекст исследования

Как уже было сказано, классические алгоритмы обучения с подкреплением требуют жестко заданной архитектуры нейросети. Чтобы избавиться от этой рутины, целесообразно использовать нейроэволюционный алгоритм NEAT. Имитируя природную эволюцию, он забирает всю лишнюю

работу на себя.

2.2. Цель исследования

Провести сравнительный анализ классического глубокого Q-обучения и нейроэволюционного подхода.

2.3. Задачи исследования

1. Реализовать базовую модель обучения с подкреплением для решения задачи моделирования поведения когнитивного агента.
2. Применить алгоритм NEAT к той же задаче для эволюции архитектуры нейросети агента.
3. Провести сравнительный анализ моделей по метрикам, определенным ниже.

2.4. Метрики исследования

Определение 2.1. *Скорость обучения определяет, насколько быстро алгоритм находит решение, удовлетворяющее заданному критерию успеха.*

Из-за различий в процедурах обучения будем использовать две шкалы:

1. Количество взаимодействий со средой: Общее число тактов симуляции, потребовавшихся агенту для достижения порогового значения награды $R_{threshold}$. Используем формулу: $N_{steps} = \sum_{i=0}^E T_i$, где E – количество эпизодов/поколений до достижения успеха, T_i – длительность i -го эпизода/поколения.

2. Время обучения: Астрономическое время в минутах, затраченное на обучение до достижения критерия успеха. Используем формулу:

$$T_{train} = \sum_{i=0}^E t_i, \text{ где } t_i - \text{ время } i\text{-го эпизода/поколения.}$$

Определение 2.2. *Качество решения – эффективность итоговой стратегии агента после завершения обучения.*

Определение 2.3. *Сложность модели – сложность ее обучения с точки зрения вычислений.*

1. Количество параметров: Используем формулу: $P = \sum_{l=1}^L (n_{l-1} + 1) \times n_l$, где L – количество слоев, n_l – число нейронов в l -м слое.
2. Плотность связей: Используем формулу: $D = \frac{E_{active}}{E_{max}}$, где E_{active} – число активных связей, E_{max} – максимально возможное число связей.

Определение 2.4. *Устойчивость – способность агента сохранять ту же производительность при изменении условий среды, на которых он не обучался явно.*

1. Устойчивость к шуму наблюдений: Измеряется как процентное снижение средней накопленной награды $\bar{R}$ при добавлении гауссовского шума к входным данным агента. Используем формулу:
$$Robustness_{noise} = \frac{\bar{R}_{clean} - \bar{R}_{noisy}}{\bar{R}_{clean}} \times 100\%$$
2. Устойчивость к изменениям среды: Измеряется как процентное изменение средней накопленной награды $\bar{R}$ при модификации среды. Используем формулу:
$$Robustness_{env} = \frac{\bar{R}_{original} - \bar{R}_{modified}}{\bar{R}_{original}} \times 100\%$$

2.5. Математическая формализация

Среда моделируется как Марковский процесс принятия решений (МППР) $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$, где $\mathcal{S}$ – пространство состояний среды, $\mathcal{A}$ – пространство действий агента, $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ – функция награды. Задача обучения с подкреплением формулируется как максимизация ожидаемого кумулятивного вознаграждения:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right] \quad (2.1)$$

где π_θ – политика агента, параметризованная вектором θ , $\gamma \in [0, 1]$ – коэффициент дисконтирования, T – максимальная длина эпизода, $\tau = (s_0, a_0, s_1, a_1, \dots, s_{T-1}, a_{T-1}, s_T)$ – траектория взаимодействия агента со средой. Подробнее МППР рассматривается в статьях [1] и [2]. В случае текущей постановки МППР мы как раз ссылаемся на [1]

2.6. Сравнимые подходы

1. Классический RL-метод: Оптимизация параметров θ нейросети фиксированной архитектуры методом градиентного спуска/подъема на основе оценки $\nabla_\theta J(\theta)$
2. Нейроэволюционный подход: Совместная оптимизация топологии сети $\mathcal{T}$ и весов θ эволюционным алгоритмом, минимизирующим сложность при максимизации $J(\theta, \mathcal{T})$

2.7. Ожидаемые результаты

Оценка преимуществ и недостатков каждого подхода в соответствии с предложенными метриками, рекомендации по выбору метода в зависимости от характеристик задачи моделирования когнитивных агентов.

3. Предшествующие работы

3.1. Глубокое обучение с подкреплением на основе аппроксимации Q-функции

3.1.1. Постановка задачи и мотивация

В фундаментальной работе [2] коллектив авторов из DeepMind (V. Mnih et al.) исследует проблему создания универсального когнитивного агента, способного обучаться сложным стратегиям управления напрямую из высокоразмерных сенсорных данных (в данном случае — необработанных пикселей экрана).

До появления этой работы классические алгоритмы обучения с подкреплением успешно применялись лишь в средах с небольшим набором дискретных состояний. При попытке перенести их на сложные задачи (где состояние среды описывается тысячами пикселей) возникало «проклятие размерности». Для решения этой проблемы традиционно требовалось ручное конструирование признаков, что лишало агента универсальности. Попытки же объединить глубокие нейронные сети и обучение с подкреплением сталкивались с серьезной проблемой нестабильности и расходимости

алгоритма из-за сильной корреляции последовательных состояний и постоянного изменения распределения данных в процессе обучения.

3.1.2. Основные идеи и архитектура метода

Для преодоления барьера между высокоразмерным визуальным входом и обучением стратегии авторы предложили алгоритм Deep Q-Network. В его основе лежит использование сверточной нейронной сети для аппроксимации функции оптимальной ценности действий $Q^*(s, a)$, которая оценивает максимальную ожидаемую награду при выборе действия a в состоянии s .

Чтобы стабилизировать обучение нейросети методами градиентного спуска в условиях RL, авторы добавили два критически важных механизма:

1. Механизм воспроизведения опыта: В процессе взаимодействия со средой агент сохраняет свои переходы (состояние, действие, награда, следующее состояние) в специальный кольцевой буфер. Обучение нейросети происходит не на последних полученных кадрах, а на случайных мини-батчах, извлеченных из этого буфера. Это разрушает временную корреляцию между обучающими примерами и сглаживает изменения в распределении данных, предотвращая деградацию накопленных знаний.
2. Пошаговое обновление целевой сети: фиксируется копия нейросети и относительно нее происходит вычисление функции ошибок. Веса основной сети обновляются на каждом шаге, а веса целевой сети копируются из основной лишь раз в C шагов. Это нужно для того, чтобы избавиться от проблемы некорректных значений функции потерь, когда веса обеих моделей, участвующих в вычислении функции

потерь, меняются на каждом этапе обучения.

3.1.3. Результаты и выявленные ограничения

Алгоритм DQN был протестирован на 49 играх платформы Atari 2600. Основным результатом стало то, что одна и та же архитектура нейросети с неизменными гиперпараметрами смогла достичь или превзойти уровень профессионального игрока-человека в большинстве игр.

Несмотря на это, глубокое обучение с подкреплением на базе DQN имеет ряд ограничений:

- Сложность оптимизации: Процесс крайне чувствителен к шагу обучения и параметрам исследования среды.
- Высокие требования к выборке: Агенту требуются миллионы тактов симуляции для схождения к оптимальной политике, так как обучение ”с нуля” через градиентный спуск идет крайне медленно на ранних этапах.

Не стоит забывать и об ограничениях, оговоренных ранее.

3.1.4. Связь с задачей моделирования когнитивных агентов

Статья [2] демонстрирует, как классический подход решает задачу моделирования когнитивного агента через градиентную аппроксимацию Q-функции при фиксированной топологии мозга нейросети.

Анализ ограничений DQN формирует четкое обоснование исследовательских задач, поставленных в п. 2.3. В то время как классический RL тратит огромные вычислительные ресурсы на обновление весов в избыточ-

ной, вручную заданной нейросети, алгоритмы семейства NEAT предлагают альтернативный путь. NEAT решает проблему жесткой архитектуры, начиная с минимально возможной топологии и инкрементально усложняя ее только там, где это необходимо для выживания агента. Сравнение метрик (скорости схождения, количества параметров и устойчивости) между подходом, вдохновленным DQN, и алгоритмом NEAT является центральным элементом практической части данной работы.

3.2. Эволюция нейронных сетей через расширение топологий

3.2.1. Постановка задачи и мотивация

В своей работе [3] Stanley и Miikkulainen рассматривают нейроэволюцию как альтернативу классическим методам обучения с подкреплением, где вместо оценки функции ценности или градиентов политики напрямую реализуется эволюция поведения агента, заданного параметрами нейросети. Ключевой вопрос: можно ли получить преимущество, эволюционируя топологию сети вместе с весами, по сравнению с традиционными подходами, где оптимизируются только веса при фиксированной архитектуре.

Авторы указывают несколько проблем в классических подходах по эволюции топологии и весов в нейронных сетях:

1. Проблема соперничающих соглашений — одна и та же функция может быть закодирована разными перестановками скрытых нейронов, что приводит к деструктивному эффекту при скрещивании. Решение этой проблемы будет рассмотрено далее.

2. Сложность сохранения инноваций: новые структурные элементы (нейроны/связи) сначала ухудшают приспособленность, в результате чего агенты быстро вымирают без специальной защиты.
3. Отсутствие механизма, который бы поощрял минимальные архитектуры и рост сложности “по мере необходимости”.

3.2.2. Основные идеи и архитектура NEAT

Проблема соперничающих соглашений в NEAT решается через систему исторических меток. Как только происходит структурная мутация, новому элементу графа, будь то узел или связь, присваивается глобальный номер инновации. Во время кроссинговера алгоритм использует эти номера как маркеры для точного выравнивания родительских геномов, что позволяет безошибочно сопоставлять общие участки и изолировать уникальные гены. Благодаря этому скрещивание не ломает уже сформированные рабочие подструктуры нейросети. Еще один важный механизм — защита инноваций. Поскольку свежие структурные изменения часто приводят к кратковременному падению эффективности, алгоритм опирается на видообразование. Для этого вычисляется метрика генетической дистанции¹, зависящая от разницы в весах и количества несовпадающих генов. Разделение популяции на виды и сохранение части агентов в этих видах помогает увеличить вероятность выживания для тех, кто еще не показал высоких результатов.

Также подробно описаны два типа структурных мутаций:

- Добавление связи между двумя ранее не связанными узлами;

¹Будет рассмотрено далее

- Расщепление существующей связи добавлением нового скрытого узла (при этом старая связь деактивируется, а новые получают веса, минимизирующие резкое ухудшение функции).

3.2.3. Связь с задачей моделирования когнитивного агента

Результаты Stanley и Miikkulainen [3] обосновывают выбор NEAT как базового нейроэволюционного алгоритма для моделирования когнитивного агента:

- NEAT естественно работает в задачах обучения с подкреплением, где поведение агента задаётся нейросетью.
- NEAT хорошо справляется с задачами с частичной наблюдаемостью и памятью (за счёт рекуррентных связей и эволюции структуры).
- NEAT позволяет избежать ручного подбора архитектуры, что является серьёзной проблемой в классическом RL-подходе, где архитектура фиксируется заранее.

Это даёт чёткую исследовательскую гипотезу:

”Эволюция топологии нейросети когнитивного агента методом NEAT может обеспечить более высокую эффективность обучения (скорость, устойчивость, компактность модели) по сравнению с классическим RL-подходом при использовании фиксированной архитектуры сети.”

4. Ход исследования

Прежде чем переходить к описанию экспериментов в симулируемых средах, необходимо формализовать алгоритмический жизненный цикл ней-

роэволюционного подхода. В данном разделе подробно рассматривается внутренняя архитектура алгоритма NEAT и механизмы его работы, которые применялись при обучении когнитивных агентов.

4.1. Алгоритмический базис: Жизненный цикл NEAT

В отличие от классического обучения с подкреплением, где оптимизируются только веса заранее заданной графовой структуры, NEAT управляет популяцией агентов, одновременно изменяя как веса, так и саму топологию графа. Процесс обучения представляет собой циклический эволюционный алгоритм, состоящий из нескольких ключевых этапов.

4.1.1. Кодирование нейросети: генотип и фенотип

Основополагающим принципом NEAT является разделение графа нейронной сети на генотип (информационный код, подверженный мутациям) и фенотип (рабочая нейронная сеть, управляющая когнитивным агентом в среде).

Генотип особи в NEAT состоит из двух списков генов:

- Гены узлов: определяют входы, выходы и скрытые нейроны агента. Каждый узел может иметь свою индивидуальную функцию активации (например, ReLU, Sigmoid или Tanh), что позволяет алгоритму подбирать оптимальную нелинейность для конкретной задачи.
- Гены связей: описывают направленные синапсы между узлами. Каждый ген связи содержит информацию о начальном и конечном узле, текущем весе, статусе активности и, что наиболее важно, номер ин-

новации.

Компиляция генотипа в фенотип происходит каждый раз, когда агента нужно протестировать в симуляции. Генотип подвергается мутациям и скрещиванию, а затем компилируется в конкретную архитектуру нейросети, которая управляет поведением агента в среде. Этот фенотип может быть как простой однослойной сетью, так и сложной рекуррентной архитектурой с несколькими скрытыми слоями и различными функциями активации.

4.1.2. Отказ от слоистой архитектуры в пользу произвольных графов

Важным концептуальным отличием алгоритма NEAT от традиционных методов глубокого обучения является полный отказ от идеи "слоев". В классических искусственных нейронных сетях, таких как многослойный перцептрон, архитектура строится строго послойно: сигнал синхронно передается от входного слоя к первому скрытому, затем ко второму и так далее, вплоть до выходного слоя. Вычисления при этом производятся структурными блоками через операции умножения матриц.

Такой отказ наделяет когнитивного агента новыми возможностями. Во-первых, один сенсор может быть соединен с моторным выходом напрямую, тогда как сигнал от соседнего датчика будет обрабатываться через сложную цепочку промежуточных узлов. Во-вторых, теперь глубина сети определяется не количеством заранее заданных слоев-матриц, а длиной максимального пути в графе от входа к выходу. Также, масштабирование графа происходит точно. Когда алгоритм принимает решение об усложнении архитектуры, он не генерирует целый скрытый слой с массой избыточных нейронов, а лишь аккуратно расщепляет одну конкретную связь, добавляя единич-

ный вычислительный узел именно в ту часть графа, где остро требуется дополнительная нелинейность.

4.1.3. Скрещивание и решение проблемы «соперничающих соглашений»

Скрещивание является одним из наиболее сложных операторов в нейро-эволюции. Основная трудность классических подходов заключалась в так называемой проблеме соперничающих соглашений, проиллюстрированная на рисунке 4.1. Она возникает, когда две нейронные сети кодируют одну и ту же функцию разными способами (например, разным расположением скрытых нейронов). При обычном случайном скрещивании таких сетей информация в геноме «перемешивается» деструктивно, что приводит к потере функциональных блоков и резкому падению приспособленности потомков.

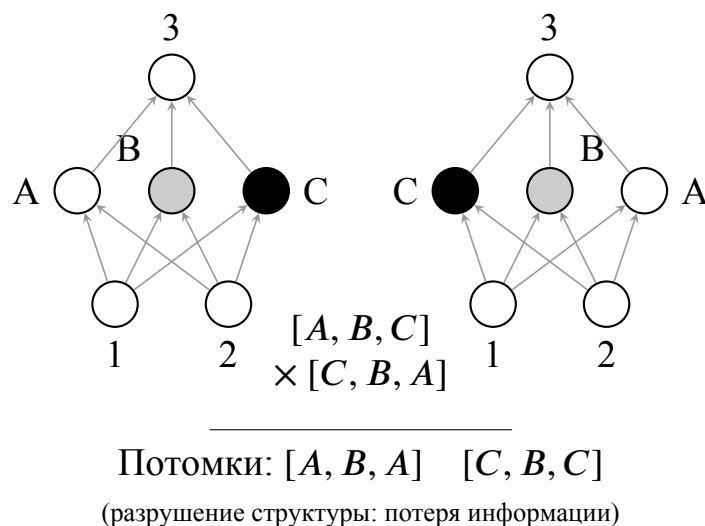


Рис. 4.1: Проблема конкурирующих соглашений

Алгоритм NEAT решает эту проблему с помощью исторических меток. Каждому новому гену (связи или узлу), появившемуся в результате

структурной мутации, присваивается уникальный идентификатор, который сохраняется за этой инновацией во всей популяции. Это позволяет алгоритму «выравнивать» геномы двух родителей, точно определяя, какие части их «цифрового мозга» имеют общее происхождение, а какие являются уникальными разработками конкретной эволюционной ветки.

4.1.4. Классификация генов при выравнивании

При создании потомка от двух родителей алгоритм сравнивает их списки генов связей. Гены распределяются по трем категориям на основе их номеров инноваций:

1. Совпадающие гены: Гены с одинаковыми номерами инноваций у обоих родителей. Это означает, что данные связи были унаследованы от общего предка. Потомок получает этот ген от одного из родителей случайным образом, а веса связей усредняются или также выбираются случайно.
2. Разъединенные гены: Гены, которые присутствуют только у одного из родителей, но их номер инновации находится внутри диапазона номеров другого родителя. Это инновации, которые произошли в прошлом одной ветви, но отсутствуют в другой из-за того, что та ветвь пошла по иному пути развития.
3. Избыточные гены: Гены, которые присутствуют только у одного из родителей и имеют номера инноваций, превышающие максимальный номер другого родителя. Это самые свежие структурные изменения, появившиеся совсем недавно в процессе эволюции одной из ветвей.

Допустим, мы скрещиваем Родителя А, содержащего гены с маркерами 1, 2, 3, 8, 10 и Родителя Б с генами 1, 2, 5, 6. Алгоритм моментально определяет, что гены 1 и 2 являются совпадающими — они достались агентам от общего предка. Гены 3, 5 и 6 классифицируются как разъединенные, поскольку их номера не превышают максимальные индексы противоположных родителей (8 и 6 соответственно), но пары для них отсутствуют. В свою очередь, гены 8 и 10 у Родителя А однозначно распознаются как избыточные, так как они выходят за верхнюю границу инноваций Родителя Б.

4.1.5. Вычисление функции приспособленности

В контексте моделирования когнитивных агентов функция приспособленности напрямую эквивалентна кумулятивной награде $J(\theta)$ из математической формализации RL (формула 2.1).

На каждом поколении для каждой особи в популяции выполняется следующий алгоритм:

1. Из генотипа компилируется фенотип (нейронная сеть прямого пространства или рекуррентная сеть).
2. Агент, управляемый этой сетью, помещается в среду (например, CartPole или LunarLander из фреймворка [4]).
3. Агент взаимодействует со средой до наступления терминального состояния (успех, провал или достижение лимита шагов).
4. Суммарная накопленная награда, полученная от среды, присваивается особи в качестве её показателя приспособленности.

Именно этот показатель определяет вероятность особи передать свои гены следующему поколению.

Для того чтобы предотвратить накопление «мусорных» структур от неудачных особей, в NEAT применяется строгое правило: разъединенные и избыточные гены наследуются только от более приспособленного родителя, имеющего более высокий показатель приспособленности. Если же родители имеют равную приспособленность, то уникальные гены передаются потомку от обоих родителей с некоторой вероятностью.

4.1.6. Операторы эволюции: мутация и скрещивание

За внесение новых генетических признаков в популяцию отвечают мутации. Чтобы алгоритм мог независимо развивать как параметры, так и саму структуру сети, этот механизм разделен на два принципиально разных класса.

Первый класс — параметрические мутации. Они не меняют архитектуру, а работают с уже существующим графом. С заданной вероятностью алгоритм либо слегка сдвигает веса синапсов, добавляя случайное значение из нормального распределения, либо полностью заменяет их на новые числа. Сюда же относится случайная смена функции активации у отдельного узла.

Второй класс — структурные мутации, являющиеся главным двигателем усложнения топологии. Здесь работают два оператора. Оператор добавления связи просто выбирает два случайных узла, между которыми еще нет прямого контакта, и прокладывает новый синапс. Это открывает новые маршруты для сигнала. Алгоритм берет активную связь, отключает

её и ставит на её место новый скрытый нейрон. Входящая к нему связь получает вес 1.0, а исходящая наследует вес старой отключенной связи. Благодаря этому в первый момент после мутации выход сети не меняется, но у алгоритма появляется новый свободный параметр для оптимизации.

4.1.6.1. Скрещивание геномов позволяет комбинировать удачные структурные блоки от разных особей. При создании потомка от двух родителей алгоритм должен корректно объединить сети, которые могут иметь совершенно разную топологию.

Здесь важную роль играют исторические маркеры. Алгоритм выстраивает гены родителей в ряд и производит выравнивание:

1. Совпадающие гены, имеющие одинаковые номера инноваций у обоих родителей, наследуются потомком случайным образом от отца или от матери.
2. Уникальные гены, присутствующие только у одного из родителей, не смешиваются вслепую. Они передаются потомку только в том случае, если принадлежат более успешному родителю (с более высоким показателем приспособленности). Если оба родителя одинаково успешны, уникальные гены могут быть унаследованы от обоих.

Такой подход гарантирует, что потомки будут накапливать полезные топологические инновации, не разрушая при этом уже сформированные функциональные блоки своих предков.

4.1.7. Видообразование на практике

Новые структурные элементы часто кратковременно снижают приспособленность особи, из-за чего в стандартных эволюционных алгоритмах инновации быстро вымирают. Для защиты структурных инноваций NEAT разбивает популяцию на виды.

Разделение происходит на основе метрики генетической дистанции δ , которая вычисляется по формуле:

$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 \cdot \overline{W} \quad (4.1)$$

где E — количество excess-генов, D — количество disjoint-генов, $\overline{W}$ — среднее различие в весах совпадающих генов, N — нормализующий фактор (обычно принимается равным размеру большей из двух родительских сетей), а c_1, c_2, c_3 — коэффициенты, регулирующие важность каждого компонента.

Если дистанция δ между особью и представителем вида меньше заданного порога $\delta_{threshold}$, то особь зачисляется в этот вид. Далее применяется механизм разделения приспособленности: скорректированная приспособленность особи рассчитывается путем деления её реальной приспособленности на размер её вида.

Благодаря этому когнитивные агенты конкурируют в основном внутри своих видов. Это не позволяет одной простой, но локально успешной топологии мгновенно захватить всю популяцию, давая сложным архитектурам время на оптимизацию своих новых весов. Инкрементальный рост сложности от минимальных сетей к более сложным является ключом к

эффективности NEAT при поиске оптимального мозга агента.

4.2. Программная реализация и особенности

NEAT-модели

Для проведения вычислительных экспериментов и моделирования когнитивных агентов в данной работе используется язык программирования Python и специализированная библиотека [5] (далее, *neat-python*). Это удобный инструмент с открытым исходным кодом и строгим соответствием оригинальной архитектуре алгоритма NEAT, предложенной авторами работы [3].

Поведение алгоритма, параметры популяции и вероятности мутаций задаются централизованно через специальный конфигурационный файл. Ниже представлен анализ ключевых секций конфигурации, использовавшейся в ходе экспериментов, и их связь с теоретическим базисом нейроэволюции.

4.2.1. Секция [NEAT]

В процессе конфигурации алгоритма мы в первую очередь задали глобальные рамки. Размер популяции (`pop_size`) был зафиксирован на протяжении всего эксперимента — это дало нам оптимальный баланс: генетического разнообразия хватало для поиска решений, а вычислительные мощности стенда не уходили в перегрузку. В качестве критерия останова был определен параметр `fitness_threshold`. Как только лучший агент в симуляции пробивал этот потолок суммарной награды, задача считалась успешно решенной, и обучение автоматически прекращалось.

4.2.2. Секция [DefaultGenome]

Чтобы концепция инкрементального роста работала на практике, мы искусственно запретили алгоритму использовать скрытые слои на старте ($\text{num_hidden} = 0$). Более того, параметр $\text{initial_connection} = \text{full_direct}$ связывал сенсорные входы (num_inputs) с моторными выходами (num_outputs) напрямую на этапе инициализации агентов. В зависимости от задачи можно выбрать необходимую функцию активации.

4.2.3. Секции [DefaultSpeciesSet] и [DefaultStagnation]

Далее нам потребовалось найти баланс между развитием новых топологий и изменения весов на заранее собранных. За темп роста графа отвечали вероятности структурных мутаций: добавление или удаление связей ($\text{add_connection_prob}$, $\text{delete_connection_prob}$) и самих узлов (add_node_prob , delete_node_prob). Параллельно с этим параметрические мутации ($\text{weight_mutation_prob}$, $\text{weight_replace_prob}$, $\text{weight_perturb_probs}$) обеспечивали изменение весов связей и узлов.

4.2.4. Секции [DefaultReproduction] и [DefaultStagnation]

Отдельного внимания потребовала настройка генетической дистанции δ (в соответствии с формулой 4.1). Чтобы защитить свежие структурные изменения, мы сделали явный упор на топологию: установили высокие штрафные коэффициенты за избыточные (c_1 , параметр $\text{compatibility_excess_coefficient}$) и разъединенные гены (c_2 , $\text{compatibility_disjoint_coefficient}$). Разница же в весовых коэффициентах

(c_3 , compatibility_weight_coefficient) наказывалась слабо, разрешая агентам внутри одного вида иметь разную силу синапсов. Границей, отделяющей один вид от другого, служил жесткий порог δ_t (compatibility_threshold). Само формирование новых поколений шло по строгим правилам отбора. Мы задали процент выживших (survival_threshold), пуская в размножение только наиболее результативных членов популяции. При этом параметр elitism гарантировал, что абсолютные чемпионы перейдут в следующую эпоху вообще без искажений от случайных мутаций. Чтобы эволюция не топталась на месте, механизмы species_elitism и max_stagnation отвечали за отсечение видов, надолго застрявших в развитии.

4.3. Условия проведения экспериментов

Для проведения вычислительных экспериментов и оценки эффективности нейроэволюционных алгоритмов использовался фреймворк gymnasium — современный стандарт в области обучения с подкреплением, предоставляющий унифицированный программный интерфейс для симуляции различных физических и логических задач.

Каждая среда в данном фреймворке строго формализована в терминах марковского процесса принятия решений (МППР), что позволяет объективно сравнивать агентов, обученных методами классического RL и алгоритмом NEAT. Первым этапом тестирования стала базовая задача управления в непрерывном пространстве состояний. Все работы проводятся в стандартном Github репозитории².

²<https://github.com/Isn0v/neat-training>

4.4. Аппаратное обеспечение экспериментов

Для обеспечения прозрачности исследования и возможности воспроизведения полученных результатов необходимо зафиксировать конфигурацию вычислительного стенда, на котором производилось обучение и тестирование когнитивных агентов.

Поскольку нейроэволюционный алгоритм NEAT начинает поиск с минимальных топологий и наращивает их инкрементально, он не требует применения специализированных графических ускорителей или тензорных процессоров, в отличие от классических избыточных архитектур глубокого обучения (например, архитектуры DQN с миллионами параметров). Основная вычислительная нагрузка при работе алгоритма ложится на центральный процессор из-за необходимости параллельной компиляции сотен индивидуальных графов (фенотипов) и симуляции физической среды для каждой особи в популяции.

В связи с этим вычислительные эксперименты проводились на локальной рабочей станции со следующими аппаратными и программными характеристиками:

- Центральный процессор (CPU): Intel Core i5-12500H (базовая тактовая частота 2.5 ГГц, поддержка многопоточных вычислений на 12 ядрах для параллельной оценки приспособленности в независимых эпизодах).
- Оперативная память (RAM): 16 ГБ, что полностью удовлетворяет требованиям к хранению истории мутаций, буферов воспроизведения опыта (в случае RL-baseline) и объектов среды *gymnasium*.

- Операционная система: Семейство Linux (Ubuntu 24.04 LTS), обеспечивающее стабильную работу с системными прерываниями и минимальные накладные расходы при аллокации памяти для процессов симуляции.
- Программная среда: Язык программирования Python 3.12.3, фреймворк для симуляции физических сред *gymnasium*, а также открытая библиотека *neat-python* для реализации эволюционного цикла.

4.5. Среда CartPole

CartPole — это классическая задача теории управления, описанная в [6]. Она представляет собой тележку, которая может двигаться по одномерному треку без трения. К каретке с помощью шарнира прикреплен неустойчивый шест, центр тяжести которого находится выше точки опоры. Задача когнитивного агента заключается в том, чтобы не дать шесту упасть, прикладывая импульсы силы к каретке.

С точки зрения когнитивного моделирования, сложность задачи состоит в том, что агент изначально не знает законов физики (гравитации, инерции, массы объектов). Он должен самостоятельно сформировать внутреннее представление динамики системы исключительно на основе поступающих числовых сигналов.

4.5.1. Пространство состояний

Сенсорный аппарат агента крайне ограничен. В каждый момент времени t среда передает агенту вектор S_t , состоящий ровно из 4 непрерывных

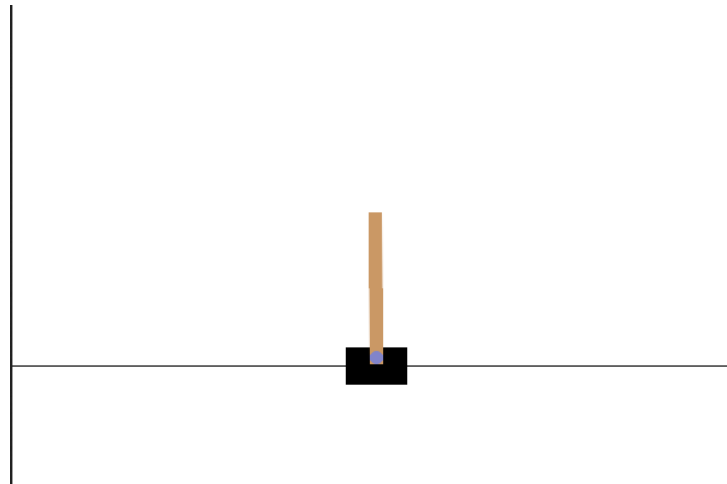


Рис. 4.2: Вид среды CartPole.

кинематических значений:

- Позиция каретки: Текущее положение тележки на треке (от -4.8 до 4.8 условных единиц). Такие значения обусловлены тем, что в оригинальном физическом эксперименте длина направляющего трека, по которому ездит тележка, составляла ровно 4.8 метра. Поскольку начальная позиция тележки находится ровно посередине трека, физический предел движения влево и вправо составляет ровно ± 2.4 метра. Этот предел был увеличен в два раза для того, чтобы среда успела зафиксировать окончательные значения при выходе агента за границы среды.
- Скорость каретки: Линейная скорость перемещения тележки.
- Угол отклонения шеста: Текущий угол наклона маятника относительно идеальной вертикали (в радианах).
- Угловая скорость шеста: Скорость падения или выравнивания маятника.

Именно этот вектор подается на входные узлы нейронной сети в про-

цессе оценки приспособленности.

4.5.2. Пространство действий

Пространство возможных решений агента дискретно. На каждом шаге агент обязан выдать одну из двух команд:

- 0: приложить фиксированный импульс силы, толкнув каретку влево.
- 1: приложить фиксированный импульс силы, толкнув каретку вправо.

Важной особенностью является отсутствие действия «ничего не делать». Агент вынужден постоянно совершать микрокорректировки, что требует от нейросети быстрой реакции и точного балансирования между противоположными командами.

4.5.3. Функция награды и условия завершения

Награда в этой среде максимально проста: агент получает +1 за каждый такт времени, в течение которого шест остается в допустимом вертикальном положении.

Эпизод симуляции считается завершенным при выполнении любого из следующих условий:

1. Падение: Угол отклонения шеста θ превысил допустимый порог (как правило, $\pm 12^\circ$ или ≈ 0.2095 радиан от вертикали). Это ограничение основано на линейном приближении синуса малых углов.
2. Выход за границы: Каретка уехала слишком далеко от центра экрана (позиция $x < -2.4$ или $x > 2.4$).

3. Успех: Достигнут лимит времени симуляции (в версии среды CartPole-v1 этот лимит составляет 500 шагов).

Агент должен достичь значения функции приспособленности в 500. Это означает, что нейросеть агента сформировала устойчивую стратегию балансировки, способную в рамках лимита среды поддерживать равновесие.

4.5.4. Конфигурация алгоритма NEAT для среды CartPole

Ниже приведены ключевые параметры, которые мы использовали в экспериментах:

4.5.4.1. Начальная топология и архитектура В соответствии с принципом постепенного усложнения структуры мы инициировали топологию нейросети с минимально возможной. Сеть не содержала скрытых слоев (`num_hidden = 0`). Входной слой состоял из 4 нейронов, принимающих вектор состояния среды: позицию каретки, ее линейную скорость, угол отклонения шеста от вертикали и его угловую скорость. Выходной слой включал 2 нейрона, аппроксимирующих уверенность агента в необходимости приложения силы влево или вправо.

На старте эволюции мы применили также полную связность прямого распространения (`initial_connection = full_direct`). Выбор итогового дискретного действия агента осуществлялся путем вычисления максимума аргумента по вектору выходных значений сети. В качестве базовой функции активации для узлов применялась функция ReLU (`activation_default = relu`).

4.5.4.2. Параметрические и структурные мутации: В процессе настройки конфигурационного файла³ мы намеренно сместили фокус на активную параметрическую адаптацию. На практике это работало так: алгоритм с высокой вероятностью (80%, параметр `weight_mutate_rate = 0.8`) корректировал веса уже существующих связей, однако сама амплитуда этих изменений жестко сдерживалась параметром `weight_mutate_power = 0.5`.

Что касается структурного роста сети, мы решили отдать приоритет уплотнению связей, а не генерации новых узлов. Логика здесь проста: алгоритм должен по максимуму выработать потенциал текущей размерности графа, прежде чем наращивать его вычислительную сложность. Именно поэтому шанс образования нового синапса составил 50% (`conn_add_prob = 0.5`), тогда как внедрение дополнительного скрытого нейрона допускалось значительно реже — лишь с вероятностью 20% (`node_add_prob = 0.2`).

4.5.4.3. Параметры популяции и критерии сходимости: Размер эволюционирующей популяции мы задали на уровне 50 особей в каждом поколении (`pop_size = 50`). Оценка приспособленности заключалась в суммарной награде, получаемой агентом за удержание маятника без падения и выхода за границы допустимой зоны.

В качестве критерия раннего останова и успешного завершения обучения мы установили порог приспособленности `fitness_threshold = 500`, что соответствует максимально возможной оценке в стандартной спецификации среды CartPole-v1. Максимальный горизонт эволюции был ограничен

³<https://github.com/Isn0v/neat-training/blob/master/environments/cartpole/neat/neat.cfg>

50 поколениями, однако, как показали дальнейшие эксперименты, алгоритм успешно находил оптимальную топологию задолго до исчерпания этого лимита.

4.5.5. Архитектура и гиперпараметры агента DQN для среды CartPole

Программная реализация алгоритма Deep Q-Network выполнялась с использованием библиотеки глубокого обучения PyTorch.

4.5.5.1. Архитектура нейронной сети: Чтобы аппроксимировать функцию ценности действий, мы спроектировали максимально компактную полносвязную сеть прямого распространения. На вход подается вектор из четырех элементов, описывающий текущую кинематику системы: координаты и скорости каретки, а также самого шеста.

Скрытый слой мы намеренно ограничили всего 16 нейронами с функцией активации ReLU, потому что пространство состояний в среде CartPole-v1 довольно простое, и небольшая размерность сети не только спасает модель от переобучения, но и ощутимо ускоряет вычислительные проходы прямого и обратного распространения ошибки. На выходе формируются сигналы от двух нейронов с линейной активацией — они выдают предсказанные Q-значения для каждого из доступных действий (толчок влево или вправо). Итоговое решение агент принимает, просто выбирая аргумент максимального значения из этого вектора.

4.5.5.2. Стратегия исследования и дисконтирование: Для соблюдения баланса между исследованием среды и эксплуатацией уже проверенных стратегий мы опирались на классическую ϵ -жадную политику. На старте параметр исследования был выставлен в максимальное значение, равное 1. По мере обучения доля случайных действий плавно срезалась: на каждом шаге применялся коэффициент затухания $\epsilon_{decay} = 0.995$, пока исследование не упиралось в минимальный порог $\epsilon_{min} = 0.01$. Дополнительно мы задали высокий фактор дисконтирования ($\gamma = 0.99$). Он обеспечивал алгоритму необходимую «дальновидность», заставляя придавать огромный вес будущим потенциальным наградам, что является критическим условием для долгосрочной балансировки маятника.

4.5.5.3. Буфер памяти и процесс оптимизации: Для стабилизации процесса обучения нейронной сети использовался механизм воспроизведения опыта. Взаимодействия агента со средой сохранялись в циклический буфер памяти емкостью 10 000 переходов. На каждом шаге обучения (после накопления достаточного объема данных) из буфера случайным образом сэмплировался мини-батч размером 64 перехода. Это позволило разорвать временную корреляцию между последовательными состояниями и свести к минимуму дисперсию градиентов при обновлении весов.

Обновление весовых коэффициентов сети осуществлялось с помощью оптимизатора Adam со скоростью обучения 0.001. В качестве целевой метрики использовалась среднеквадратичная ошибка. Минимизируемая функция потерь L строилась на основе отклонения текущего предсказания сети от целевого значения, вычисляемого согласно уравнению Беллмана:

$$L = \frac{1}{N} \sum_{t=0}^T ((r_t + \gamma \max_{a'} Q(s_t, a')) - Q(s_t, a_t))^2 \quad (4.2)$$

где N – размер батча, s_t и a_t – текущие состояние и действие, r_t – полученная награда, s'_t – следующее состояние, а γ – коэффициент дисконтирования.

Критерием успешного решения среды служило достижение суммарной награды в 500 баллов за один эпизод, что означает непрерывное удержание шеста в течение 500 тактов симуляции. По достижении этого порога веса модели (state_dict) автоматически сохранялись для последующей демонстрации и анализа.

4.6. Среда LunarLander

Среда LunarLander впервые упоминается в пакете сред Gym [4], которая представляет собой задачу управления в условиях двумерной физической симуляции с непрерывным пространством состояний. Задача когнитивного агента заключается в осуществлении безопасной и мягкой посадки космического модуля на заданную посадочную площадку, расположенную между двумя маркерными флажками. В отличие от задачи балансировки маятника, здесь агент должен научиться преодолевать гравитацию и управлять инерцией, аккуратно дозируя тягу главного и двух маневровых двигателей.

В рамках данного исследования для оценки способности алгоритмов к обобщению устойчивости тестирование проводилось в двух различных конфигурациях среды: в условиях идеального вакуума и с применением сильного бокового ветра и турбулентности. Сравнительный анализ влияния

этих возмущающих факторов на стабильность агентов подробно рассматривается в разделе экспериментальных результатов.

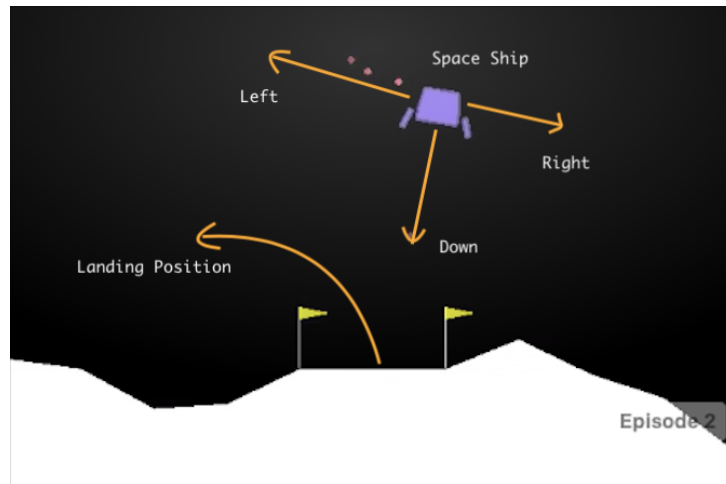


Рис. 4.3: Вид среды LunarLander.

4.6.1. Пространство состояний

Сенсорный аппарат агента передает вектор наблюдений, состоящий из 8 числовых параметров, исчерпывающе описывающих текущее кинематическое состояние модуля:

- Позиция центра масс аппарата в двумерной системе координат (x, y) .
- Вектор линейной скорости перемещения модуля по осям x и y .
- Угол наклона (крена) аппарата относительно идеальной вертикали.
- Угловая скорость вращения модуля.
- Два дискретных булевых флага, сигнализирующих о наличии физического контакта левой и правой посадочных опор с поверхностью.

4.6.2. Пространство действий

Пространство возможных решений агента является дискретным и состоит из 4 взаимоисключающих команд, выбираемых нейросетью на каждом такте симуляции:

- 0: Не предпринимать никаких действий (свободное падение).
- 1: Включить левый маневровый двигатель (для корректировки крена вправо).
- 2: Включить главный (нижний) маршевый двигатель.
- 3: Включить правый маневровый двигатель (для корректировки крена влево).

4.6.3. Функция награды и условия завершения

В отличие от бинарного сигнала выживания в CartPole, функция приспособленности в LunarLander имеет сложную структуру, сподвигающую достижение цели при максимальной экономии ресурсов. Помимо базовых наград, агент получает весомое поощрение от 100 до 140 очков за уверенное движение к центру посадочной площадки и правильное гашение скорости. Цена ошибки здесь высока, поскольку крушение с падением на корпус мгновенно накладывает штраф в 100 баллов. Чтобы научить сеть экономить топливо, мы также внедрили пенальти за работу двигателей. Каждый такт работы маршевого двигателя обходится в -0.3 очка, а использование боковых маневровых штрафует на -0.03 очка. Симуляция одного эпизода жестко ограничена лимитом в 1000 шагов и прерывается досрочно при

крушении или успешном касании поверхности. Если посадка прошла идеально мягко и точно в центр, агент способен заработать порядка 250–300 очков. При этом алгоритм считается успешно сошедшимся, если средняя награда за эпизод стабильно закрепляется за официальным порогом среды в 100–200 баллов.

4.6.4. Конфигурация алгоритма NEAT для LunarLander

Также используем файл конфигурации NEAT⁴

4.6.4.1. Основные параметры популяции и завершения: Чтобы обеспечить эффективный поиск в пространстве решений и поддерживать достаточное генетическое разнообразие, мы остановились на размере популяции в 150 особей (`pop_size = 150`). Естественно, эволюция была направлена на максимизацию функции приспособленности (`fitness_criterion = max`). Процесс обучения автоматически останавливался при `fitness_threshold = 200.0`.

4.6.4.2. Архитектура и инициализация генома: В соответствии со спецификацией среды LunarLander мы зафиксировали 8 входных и 4 выходных узла (`num_inputs = 8`, `num_outputs = 4`). В соответствии с главной философией алгоритма NEAT, стартовая топология была максимально минималистичной — без скрытых слоев (`num_hidden = 0`). На этапе инициализации каждый сенсор соединялся с моторными выходами напрямую (`initial_connection = full_direct`). Вычислительная нелинейность достигалась за счет использования гиперболического тангенса (`activation_default`

⁴<https://github.com/Isn0v/neat-training/blob/master/environments/lunar-lander/neat/neat.cfg>

= $\tanh$) во всех узлах. Это решение подошло для управления двигателями, поскольку тангенс выдает сигналы строго в необходимом диапазоне от -1 до 1.

4.6.4.3. Вероятности мутации: Для постоянного уточнения и «тонкой настройки» уже существующих путей управления мы установили весьма высокую вероятность изменения веса связей — 80% (`weight_mutate_rate = 0.8`). Что касается роста сложности топологии, здесь шансы распределились иначе. Вероятность появления новой синаптической связи составила 50% (`conn_add_prob = 0.5`), а вот новый скрытый нейрон мог добавиться лишь с 20% вероятностью (`node_add_prob = 0.2`). Чтобы сеть не только разрасталась, но и могла избавляться от избыточных структур, мы заложили аналогичные вероятности на удаление нейронов (20%, `node_delete_prob = 0.2`) и связей (50%, `conn_delete_prob = 0.5`).

4.6.4.4. Специация и эволюция популяции: Две сети относились к разным видам, если порог их генетической совместимости δ превышал 3.0 очка (`compatibility_threshold = 3.0`). Также, если вид не демонстрировал улучшения приспособленности на протяжении 20 поколений (`max_stagnation = 20`), он признавался стагнирующим и полностью лишался ресурсов для дальнейшего размножения.

4.6.5. Архитектура и гиперпараметры агента DQN для среды LunarLander

Программная реализация алгоритма DQN для задачи посадки лунного модуля базировалась на фреймворке `stable_baselines3`, рассмотренном в работе [7]. Он предоставляет оптимизированные и надежные реализации современных алгоритмов обучения с подкреплением. В связи с необходимостью комплексной оценки устойчивости когнитивных агентов, обучение проводилось в два этапа: формирование базовой политики управления⁵ и универсальной политики⁶ в условиях ветренности.

Процесс Q-обучения регулировался следующим набором гиперпараметров:

- Буфер воспроизведения опыта: Емкость буфера составила 100 000 переходов, что обеспечило эффективную декорреляцию обучающей выборки. Процесс обучения инициировался после накопления первых 1 000 случайных шагов (`learning_starts = 1000`).
- Оптимизация и градиентный спуск: Использовался размер пакета данных равный 128. Скорость обучения была установлена на уровне 10^{-3} .
- Параметры обновления: Дисконтирующий множитель γ составил 0.99, ориентируя агента на долгосрочное планирование. Синхронизация весов целевой сети производилась каждые 250 шагов оптимизации.
- Стратегия исследования среды: Применялась ϵ -жадная стратегия с

⁵<https://github.com/Isn0v/neat-training/blob/master/environments/lunar-lander/rl/model.py>

⁶https://github.com/Isn0v/neat-training/blob/master/environments/lunar-lander/rl/model_windy.py

линейным затуханием. Начальное значение параметра исследования $\epsilon = 1.0$ постепенно снижалось до 0.05 на протяжении первых 10% от общего времени обучения (`exploration_fraction = 0.1`), после чего агент переходил к преимущественной эксплуатации полученных знаний.

4.7. Среда Ant

В качестве полигона для исследования возможностей нейроэволюции была выбрана среда непрерывного управления Ant-v4 из библиотеки Gymnasium. Данная среда представляет собой физическую симуляцию на базе движка MuJoCo, описанную в [8], для четырехногого агента («муравья»), цель которого – максимально быстро и эффективно продвигаться вперед (по оси X), сохраняя при этом баланс и минимизируя затраты энергии на лишние движения.

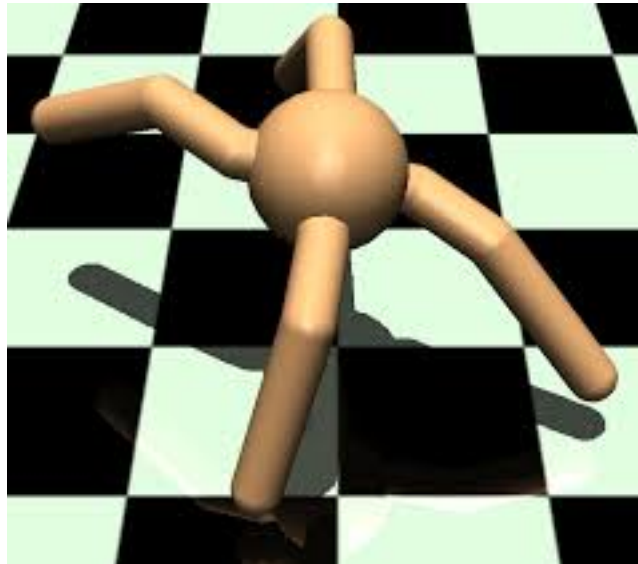


Рис. 4.4: Схематическое изображение агента в среде Ant-v4.

4.7.1. Пространство состояний и действий

Сложность среды обусловлена физическими параметрами модели:

- Пространство состояний (27 параметров): включает в себя данные о положении корпуса в пространстве, углы поворота всех восьми сочленений, а также векторы их линейных и угловых скоростей. Такой объем данных необходим нейросети для понимания текущей позы робота и инерции его частей.
- Пространство действий (8 параметров): агент управляет крутящими моментами, прикладываемыми к каждому из восьми суставов. Действия являются непрерывными и лежат в диапазоне $[-1, 1]$.

4.7.2. Функция награды и условия завершения

Функция приспособленности в данном эксперименте базируется на стандартной системе вознаграждений Ant-v4, которая включает:

1. Reward for moving forward: поощрение за продвижение вдоль оси X.
2. Healthy reward: фиксированный бонус за каждый шаг, пока робот остается «здоров» (не перевернулся и не вышел за пределы допустимых углов).
3. Control cost: штраф за слишком резкие или энергозатратные движения, что стимулирует агента к плавной и естественной походке.

Для оптимизации процесса эволюции была внедрена эвристика «жесткого таймера». Если в течение 50 шагов симуляции агент не продвигается вперед более чем на 0.05 метра, эпизод прерывается досрочно. Это позволя-

ет алгоритму быстрее переходить к оценке следующей особи, не тратя время на агентов, которые застряли или совершают хаотичные движения на месте. Окончательная оценка генома вычисляется как среднее арифметическое результатов трех независимых эпизодов.

4.7.3. Конфигурация алгоритма NEAT для среды Ant-v4

Настройка⁷ алгоритма NEAT для среды Ant-v4 оказалась непростой задачей. Главная сложность заключалась в высокой размерности пространства действий: малейшие хаотичные скачки в топологии растущей сети могли легко разрушить только начавший формироваться навык ходьбы агента. Чтобы взять этот процесс под контроль, мы разделили архитектуру решения на три обособленных блока: базовые архитектурные параметры, генетические механизмы и саму вычислительную инфраструктуру.

4.7.3.1. Архитектурные параметры и инициализация: Стартовую архитектуру нейросетей мы также сделали примитивной. Размерность слоев мы жестко привязали к физической модели «муравья»: сеть получила 27 сенсорных узлов на входе и 8 моторных на выходе.

Как и в предыдущих экспериментах, мы полностью отказались от использования скрытых нейронов на старте. Все сенсорные данные напрямую пробрасывались на выходы. Подобный подход давал эволюции возможность сперва отточить базовые линейные зависимости в управлении роботом, и лишь затем, по мере необходимости, выстраивать сложные нелинейные цепи.

⁷<https://github.com/Isn0v/neat-training/blob/master/environments/ant/neat.cfg>

Для выходных сигналов идеально подошел гиперболический тангенс, потому что он совпадает с требованиями физического движка MuJoCo к управляющим импульсам. При этом классическое суммирование входящих сигналов внутри узлов мы заменили на усреднение.

4.7.3.2. Генетические операторы и мутации:

- Генетические операторы: Для обеспечения баланса между эксплуатацией текущих решений и исследованием новых топологий были установлены следующие коэффициенты:
- Мутация весов и смещений: используется гауссово распределение со средним значением 0.0 и стандартным отклонением 0.1. Максимальные и минимальные значения весов ограничены диапазоном $[-3.0, 3.0]$ для предотвращения «взрыва» активаций в глубоких слоях эволюционирующей сети.
- Структурные мутации: вероятность добавления новой связи (`conn_add_prob`) составляет 0.3, а добавления нового узла (`node_add_prob`) – 0.1. При этом вероятности удаления компонентов (`conn_delete_prob`, `node_delete_prob`) установлены в 0, что гарантирует сохранение накопленной сложности топологии в рамках данного эксперимента. Данное решение было принято для ускорения сходимости, так как большинство полезных мутаций в Ant-v4 связано с добавлением новых связей, а не удалением существующих.

4.7.3.3. Видообразование и репродукция: Порог генетической совместимости был зафиксирован на отметке 1.2. На практике это позволило

аккуратно дробить обширную популяцию из 500 особей на изолированные, устойчивые группы с похожими топологиями сетей. Внутри каждого сформированного вида гарантированно выживала только одна лучшая особь, тогда как для популяции в целом этот лимит составлял 5 агентов, набравших наибольший результат приспособленности. Право участвовать в кроссинговере и передавать свои гены следующему поколению получали лишь 20% наиболее результативных агентов.

4.7.3.4. Инфраструктура обучения и отказоустойчивость: Перейдем к технической реализации, находящейся в файле `model.py`⁸.

Вычисления в среде Ant-v4 ресурсоемки, поэтому нам пришлось внедрить ряд решений для обеспечения стабильности экспериментов. Во-первых, мы ушли от последовательного выполнения кода и распределили расчет приспособленности агентов на все доступные ядра процессора с помощью инструмента `ParallelEvaluator`. Такая параллелизация позволила радикально ускорить симуляцию, сократив время обработки одного поколения примерно в 4–8 раз в соответствии с количеством физических ядер устройства. Во-вторых, мы учли тот факт, что эволюция многосуставного робота — это тяжелый процесс, который может длиться от нескольких часов до нескольких дней. В таких условиях очень важной оказалась отказоустойчивость. Мы настроили систему чекпоинтов и теперь алгоритм автоматически делает полный срез популяции и сохраняет весь накопленный прогресс каждые 25 поколений. Это надежно страховало нас от потери результатов в случае любых системных сбоев или переполнения

⁸<https://github.com/Isn0v/neat-training/blob/master/environments/ant/model.py>

памяти.

5. Результаты исследования

В рамках выпускной квалификационной работы было проведено исследование применимости нейроэволюционных алгоритмов для задачи моделирования поведения когнитивных агентов. Главный вектор исследования был направлен на концептуальный и эмпирический сравнительный анализ эволюционного подхода с классическими методами глубокого обучения с подкреплением.

Оценка эффективности обученных агентов производилась не только по классическому критерию максимизации получаемой награды, но и с учетом таких важных для реальных систем параметров, как архитектурная сложность нейронной сети, скорость схождения и устойчивость к шумовым воздействиям.

Полученные в ходе симуляций экспериментальные данные позволили сформулировать сильные и слабые стороны каждого из подходов, которые мы рассмотрим далее.

5.1. Результаты в среде CartPole

Первым этапом экспериментального исследования стала классическая задача управления — балансировка перевернутого маятника. Данная среда выступала в качестве фундаментального полигона для верификации работоспособности алгоритмов и оценки базовых характеристик когнитивных агентов. В рамках эксперимента проводилось сопоставление классического

алгоритма DQN и нейроэволюционного метода.

На этапе проектирования архитектур моделей были применены два принципиально разных подхода. Агент на базе DQN использовал фиксированную полносвязную нейронную сеть с одним скрытым слоем, содержащую 114 параметров. В противовес этому, алгоритм NEAT функционировал в парадигме постепенного усложнения, начиная эволюционный процесс с простейшей топологии без скрытых слоев и динамически формируя структуру сети в ответ на требования среды.

Исходя из сравнительного анализа метрик (согласно определениям 2.1-2.4) мы сделали следующие выводы:

5.1.1. Эффективность использования выборки и скорость схождения

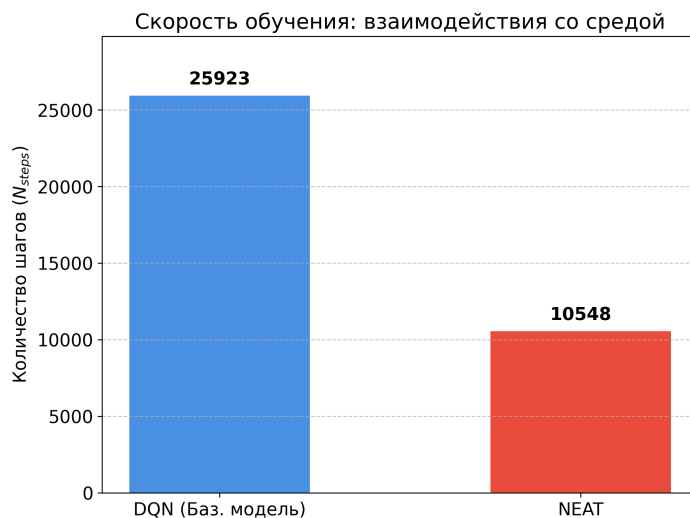


Рис. 5.1: Скорость схождения двух алгоритмов.

Оба алгоритма успешно справились с задачей, однако нейроэволюционный подход продемонстрировал существенно более высокую скорость адаптации (Рисунок 5.1). Алгоритму NEAT потребовалось всего 4 поколения (или 10 548 шагов взаимодействия со средой), чтобы найти оптимальную

стратегию. В то же время алгоритму DQN для стабилизации весов и схождения потребовалось 471 эпизод, что эквивалентно 25 923 взаимодействиям. Эволюционный алгоритм оказался более чем в 2,4 раза эффективнее с точки зрения затрат на вычисления.

5.1.2. Качество решения и архитектурная сложность

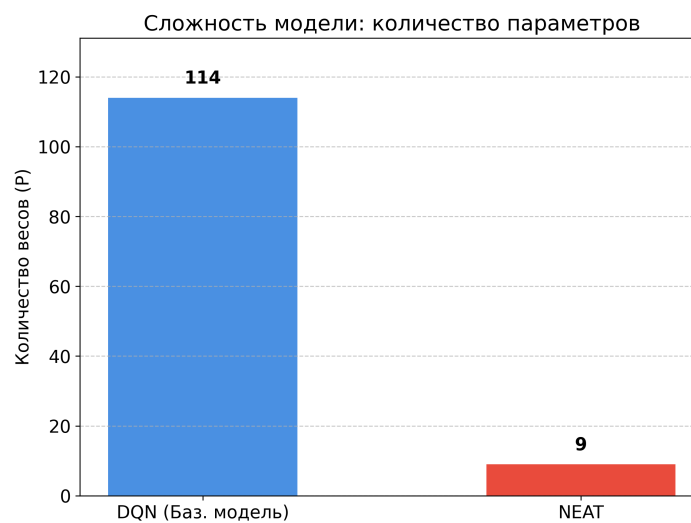


Рис. 5.2: Архитектурная сложность двух алгоритмов.

В идеальных условиях обе модели достигли абсолютного максимума качества (средняя награда 500.00 из 500) со 100% стабильностью (стандартное отклонение равно 0). Однако способы достижения этого результата кардинально различаются. На рисунке 5.2 видно, что в то время как DQN использует плотную полносвязную структуру (плотность связей 100%), NEAT сформировал высокоэффективную разреженную сеть, состоящую всего из 9 параметров (плотность связей 75%). Это доказывает способность нейроэволюции находить экстремально компактные и легковесные решения, что критически важно для развертывания моделей на устройствах с ограниченными вычислительными ресурсами.

5.1.3. Устойчивость к внешним возмущениям

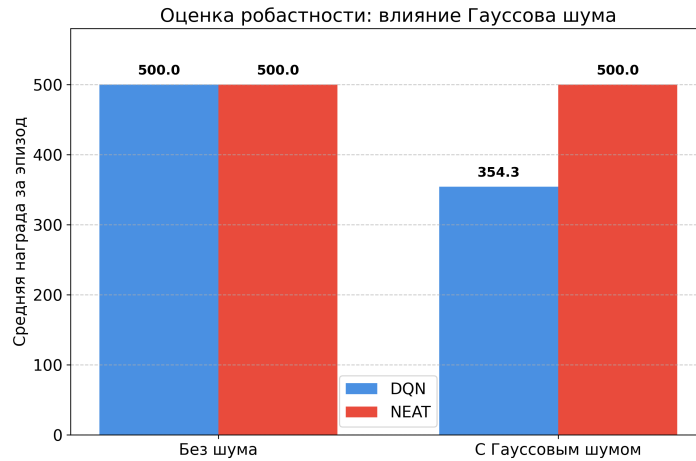


Рис. 5.3: Устойчивость двух алгоритмов.

Наиболее значимое различие между алгоритмами было зафиксировано при стресс-тестировании агентов в условиях зашумленных данных (Рисунок 5.3). Для генерации шума в этом и последующий проектах используется Гауссовский шум. В данной среде берется стандартное отклонение равное 0.1. При добавлении шума в наблюдения среды, эффективность полносвязной модели DQN резко снизилась на 29,14% (средняя награда упала до 354.30). Агент оказался склонен к переобучению под «идеализированную» картину мира. Напротив, агент NEAT продемонстрировал абсолютную устойчивость: в условиях аналогичного зашумления модель не потеряла ни одного балла, сохранив среднюю награду на уровне 500.00. Столь высокая устойчивость напрямую вытекает из компактности эволюционной архитектуры: из-за отсутствия «лишних» связей и параметров, сеть физически лишена способности пропускать через себя шумовые сигналы, фильтруя их на уровне самой топологии.

5.1.4. Практические выводы

Эксперимент в среде CartPole эмпирически доказал, что для решения базовых задач когнитивного управления нейроэволюционный алгоритм NEAT превосходит классический DQN по скорости схождения, компактности получаемой модели и, что наиболее важно, по уровню устойчивости к неопределенностям среды.

5.2. Результаты в среде LunarLander

Эксперименты в среде LunarLander стали ключевым этапом практического исследования, позволившим оценить работу когнитивных агентов в условиях сложного пространства состояний и необходимости тонкого микроконтроля. В отличие от базовой задачи CartPole, здесь перед агентами стояла цель не просто удержания баланса, а поиска оптимальной траектории посадки в условиях меняющейся гравитации и внешних возмущений.

5.2.1. Сравнительный анализ архитектурной сложности и обучения

В ходе исследования был зафиксирован значительный контраст в подходах к формированию управляющей политики:

Практика опять подтвердила гипотезу контрасте между тем, как алгоритмы формируют управляющую политику. Если смотреть на скорость схождения на рисунке 5.4, то алгоритм DQN оказался явным лидером, потому что пробил целевой порог приспособленности (>200) всего за 170 000 шагов взаимодействия со средой, что заняло около 224 секунд реального времени. Нейроэволюционному подходу потребовалось куда больше итера-

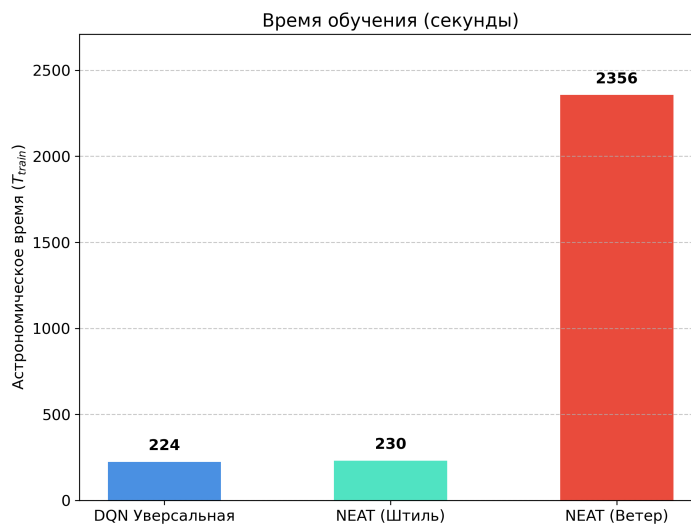


Рис. 5.4: Скорость схождения двух алгоритмов.

ций для поиска рабочей топологии. Однако эта задержка на этапе обучения полностью окупается в момент инференса. Итоговая информационная емкость моделей отличается на порядки, что можно увидеть на рисунке ??.

Для достижения стабильных результатов DQN-агенту понадобилась тяжеловесная глубокая сеть размерностью [512, 512], содержащая 69 124 обучаемых параметра. NEAT же, напротив, вырастил очень компактный граф всего из 150–200 связей и узлов.

5.2.2. Феномен «нейролоботомии» и проблема локальных оптимумов

Во время симуляций мы столкнулись с весьма специфическим побочным эффектом, который условно классифицировали как «нейролоботомию». Из-за того, что среда накладывает жесточайший штраф за крушение модуля в -100 очков, самый безопасный выход для развития адаптации агента к среде, но не менее деструктивный выход стал в том, чтобы вообще ничего не делать. Агент буквально обрывал связи в собственной сети и отключал двигатели, лишь бы не совершить фатальную ошибку при активном

маневрировании. Статистически алгоритму было выгоднее просто падать вниз, чем пытаться управлять модулем и разбиваться. Чтобы вытащить сеть из этого локального оптимума, нам пришлось перенастраивать функции приспособленности и добавлять элементы рандомизации в стартовые условия.

5.2.3. Устойчивость в условиях неопределенности и внешних шумов

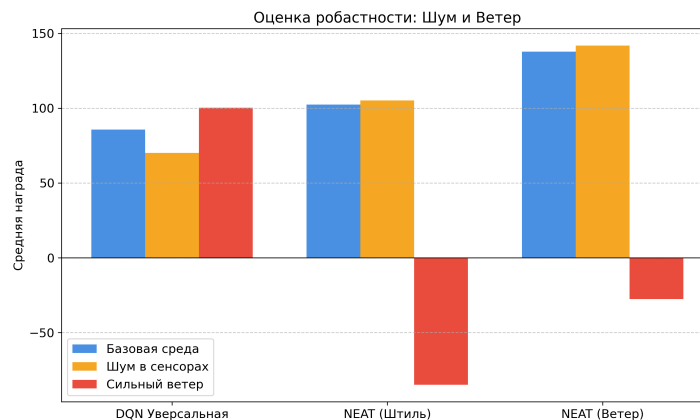


Рис. 5.5: Устойчивость двух алгоритмов.

Отдельной задачей стало стресс-тестирование агентов на устойчивость к внешним возмущениям вроде турбулентности или бокового ветра. Для DQN пришлось написать специальную обертку `UniversalWindWrapper`, которая в половине эпизодов генерировала порывы ветра мощностью до 20.0. Это вынуждало полносвязную сеть искать паттерны компенсации смещения. С алгоритмом NEAT мы поступили иначе: заставили каждый геном сдавать серию из 5 испытаний (`EPISODES_PER_GENOME = 5`), где штиль чередовался с непогодой. Итоговая оценка суммировалась, а фактор случайного везения сводился к нулю.

Сводные результаты этих тестов на рисунке 5.5 показывают, что универ-

сальная DQN-модель отлично справляется в идеальных условиях, набирая 85.75, но слаба перед шумами в сенсорах, проседая по эффективности до 70.13. В то же время NEAT показал хорошую стабильность к высокочастотным помехам из-за низкой сложности архитектуры нейросети.

5.2.4. Специфика программной реализации

Чтобы гарантировать научную достоверность и воспроизводимость всех экспериментов, мы строго контролировали генерацию случайных чисел. Это ставило всю популяцию из 150-300 агентов в абсолютно идентичные стартовые условия, делая эволюционный отбор полностью равным. А учитывая тяжелую физику LunarLander и необходимость пятикратной проверки каждой особи, мы распараллелили вычисления по ядрам процессора через модуль multiprocessing.

5.2.5. Практические выводы

Классический подход DQN — это отличный выбор для быстрого обучения в детерминированной среде с высокой размерностью данных. Но если речь заходит о создании устойчивых систем управления, которым предстоит работать в условиях неопределенности и жесткого дефицита вычислительных ресурсов, нейроэволюция оказывается гораздо более предпочтительным инструментом.

5.3. Результаты в среде Ant

Экспериментальное исследование в среде симуляции четырехногого робота стало завершающим и наиболее технически сложным этапом практической части работы. Данная среда, базирующаяся на физическом движке MuJoCo, отличается высокоразмерным непрерывным пространством состояний и действий, что предъявляет экстремальные требования к архитектуре когнитивного агента.

5.3.1. Обоснование выбора модальности обучения

В отличие от сред CartPole и LunarLander, где проводился сравнительный анализ двух алгоритмов, моделирование в среде Ant-v4 осуществлялось исключительно с использованием нейроэволюционного подхода. Данное ограничение было введено намеренно и обусловлено фундаментальными математическими ограничениями классического алгоритма глубокого DQN.

Поскольку управление «муравьем» требует подачи непрерывных сигналов на 8 независимых суставов, применение DQN потребовало бы искусственной дискретизации пространства действий. Даже при минимальном разбиении (например, на 3 состояния: тянуть назад, покой, тянуть вперед) алгоритм столкнулся бы с проблемой комбинаторного взрыва — $3^8 = 6561$ уникальная комбинация действий на каждом такте. Нейроэволюционный алгоритм NEAT, напротив, способен напрямую генерировать векторы непрерывных управляющих сигналов, что делает его оптимальным выбором для задач робототехники в рамках данного исследования.

5.3.2. Эволюция функции вознаграждения и адаптация среды

Для достижения биомеханически корректного движения нам пришлось применить метод последовательного усложнения функции вознаграждения.

Первым серьезным препятствием стал встроенный штраф за расход энергии (`ctrl_cost`). В базовой версии среды любые резкие сигналы на суставы карались настолько жестко, что этот штраф перекрывал награду за выживание. Алгоритм намеренно сводил топологию нейросети к простейшей и выдавал нули на моторы, чтобы вообще не получать штрафы. Для преодоления этого тупика мы программно снизили весовые коэффициенты штрафов в коде инициализации среды, дав агенту право на ошибку при первичных попытках моторики.

Следующей проблемой стал таймер симуляции. Изначально муравей появлялся в воздухе и падал на поверхность, а базовый код убивал агента за отсутствие скорости еще до того, как тот успевал сгруппироваться после падения. В ответ на это мы разработали эвристику умного таймера. Она предоставляла агенту льготные 150 тактов на стабилизацию и поощряла любое, даже самое незначительное продвижение вперед.

Лишь после того, как агент научился выживать на старте и базово перебирать конечностями, система вознаграждения была финально переориентирована. Фокус сместился на максимизацию скорости центра масс по оси X при плавном движении, что в итоге и привело к формированию биомеханически правильной походки.

5.3.3. Практические выводы

Благодаря поэтапной модификации функции вознаграждения, алгоритм NEAT успешно синтезировал топологию нейронной сети, способную управлять сложной многозвенной системой.

Эволюционный процесс привел к отказу от хаотичных, судорожных движений в пользу скоординированной походки. Агент научился использовать инерцию собственного тела, синхронизируя работу 8 суставов для поддержания баланса и минимизации контакта корпуса с землей. Итоговая модель нейросети оказалась поразительно компактной, продемонстрировав способность нейроэволюции находить элегантные решения в многомерных пространствах без избыточного наращивания вычислительных мощностей.

6. Направления дальнейших исследований

Опираясь на полученные в ходе симуляций экспериментальные данные и подтвержденную эффективность нейроэволюционных подходов, был намечен ряд стратегических векторов для дальнейшего развития данного проекта. В частности, планируется масштабирование разработанных методов как в сторону усложнения архитектуры агентов, так и в сторону практического применения в прикладных симуляторах.

6.1. Разработка гибридных мультимодальных моделей

Первым направлением развития является преодоление ограничений классического алгоритма NEAT при работе со средами, обладающими вы-

сокой размерностью входных данных. В ходе исследования было отмечено, что базовая нейроэволюция испытывает существенные вычислительные трудности при обработке сырых пикселей (например, при получении визуальной информации напрямую с камер агента). Прямая подача изображений на вход NEAT приводит к комбинаторному взрыву возможных топологий и критическому замедлению эволюции.

Для решения этой проблемы мы планируем разработать и протестировать гибридную мультимодальную архитектуру, в которой задачи будут жестко разделены. В роли модуля зрения выступают сверточные нейронные сети. Их задачей станет автоматическое извлечение визуальных признаков и сжатие исходной картинки до компактного информационного вектора.

Затем этот вектор будет передаваться непосредственно в алгоритм NEAT. Апробацию этой связки планируется провести на стандартизированном наборе сред с визуальным входом, например, классических играх Atari.

6.2. Интеграция алгоритмов в симулятор популяций

Второе направление представляет собой глобальную практическую цель, послужившую фундаментальной мотивацией для проведения данного дипломного исследования. В настоящее время на базе института ИСИ СО РАН ведется разработка комплексного симулятора поведения популяций виртуальных муравьев.

Внедрение алгоритмов машинного обучения в подобные масштабные проекты напрямую, «вслепую», несет высокие риски нестабильности си-

стемы. В связи с этим было критически важно провести предварительный изолированный анализ: проверить сильные и слабые стороны алгоритма NEAT на стандартизированных кинематических моделях (таких как среда непрерывного управления Ant-v4 в физическом движке MuJoCo), настроить функции вознаграждения и изучить поведение агентов вне влияния побочных факторов.

Результаты текущей работы сформировали готовую, эмпирически доказанную методологию. Мы на практике подтвердили два ключевых преимущества нейроэволюции, критически важных для симуляции популяций:

- Легковесность моделей: Компактность сформированных топологий позволяет запускать сотни и тысячи независимых агентов одновременно без перегрузки вычислительных мощностей системы (в отличие от «тяжелых» сетей DQN).
- Устойчивость к зашумлению: Разреженная архитектура сетей делает агентов робастными к изменениям внешней среды и непредсказуемым физическим взаимодействиям.

В качестве финального этапа развития проекта, протестированный и оптимизированный пайплайн обучения NEAT будет интегрирован в архитектуру институтского симулятора. Это позволит перейти от управления одиночными когнитивными агентами к моделированию сложных эмерджентных форм поведения в рамках целых популяций виртуальных насекомых.

Список литературы

1. Sutton R.S., Barto A.G. Reinforcement learning: An introduction. 2-е изд. The MIT Press, 2018 .
2. Mnih V. и др. Human-level control through deep reinforcement learning // Nature. Nature Publishing Group, a division of Macmillan Publishers Limited. All Rights Reserved., 2015. Т. 518, № 7540. С. 529–533.
3. Stanley K.O., Miikkulainen R. Evolving neural networks through augmenting topologies // Evol Comput. 2002. Т. 10, № 2. С. 99–127.
4. Towers M. и др. Gymnasium: A standard interface for reinforcement learning environments. 2025.
5. McIntyre A. и др. neat-python.
6. Barto A.G., Sutton R.S., Anderson C.W. Neuronlike adaptive elements that can solve difficult learning control problems // IEEE Trans Syst Man Cybern. 1983. Т. SMC-13, № 5. С. 834–846.
7. Raffin A. и др. Stable-baselines3: reliable reinforcement learning implementations // J. Mach. Learn. Res. JMLR.org, 2021. Т. 22, № 1. 8 с.
8. Todorov E., Erez T., Tassa Y. Mujoco: A physics engine for model-based control // Intelligent robots and systems (IROS), 2012 IEEE/RSJ international conference on. 2012. С. 5026–5033.